

RTL Design Sherpa

Lattice Generator Architecture Specification 1.2

October 14, 2026

Table of Contents

1 Lattice Gas Index....8

2 Document Information....9

2.1 Lattice Generator Architecture Specification....9

2.2 Revision History....10

2.3 Document Purpose....11

2.4 Intended Audience....12

2.5 Related Documents....13

3 What the Generator Produces....14

3.1 Output artifacts....15

3.2 Architecture in the generated module....16

4 When to Regenerate....17

4.1 Pre-generated vs custom....18

5 Generator Entry Points....19

5.1 Low-level generator....19

5.2 Convenience batch script....20

5.3 Programmatic API....21

6 CLI and Programmatic API....22

6.1 apbx_xbar_generator.py....23

6.2 generate_xbars.py....24

6.3 Programmatic API....25

7 Parameter Semantics....26

7.1 Generation-time parameters....27

7.2 Elaboration-time parameters....28
8 Mixed APB4/APB5 and Parity....29
8.1 Version list rules....30
8.2 PWAKEUP direction and forwarding....31
8.3 Mixed-port sideband behavior....33
8.4 Parity (enable_parity=True)....34
9 Validation Rules....36
9.1 Structural checks....37
9.2 Address-map checks....38
9.3 Parity checks....39
10 Pipeline Overview....40
10.0.1 Figure 3.1: Lattice Generator Pipeline....40
10.1 Stages....41
11 Request Decode....42
11.1 Offset-based slave selection....43
11.2 Decode-miss handling....44
11.3 Single-master simplification....45
12 Arbitration and Routing....46
12.1 Per-slave request vector....47
12.2 Round-robin arbitration....48
12.3 Command routing....49
12.4 Response routing....50
12.5 Ready routing....51
13 Boundary Module Instantiation....52

13.1 Master-side modules....53
13.2 Slave-side modules....54
13.3 Parity tie-offs....55
13.4 PWAKEUP wiring....56
13.5 1×1 passthrough....57
14 Directory Layout....58
15 Module Header and Parameters....59
15.1 Address-map comment....60
15.2 Port naming convention....61
16 cmd/rsp Fabric....62
16.1 Fabric width invariance....64
17 Wrapper Scaffolds....65
17.1 Filelist convention....66
18 Full Regeneration Rule....67
18.1 Regeneration command....68
18.2 Do not hand-edit generated files....69
19 Pre-Generated Fixtures....70
19.1 When to add a new fixture....71
20 DV Suite....72
20.1 Test list....73
20.2 Running the suite....74
20.3 Adding a new test....75
21 Lint Checks....76
21.1 Width checks....77

21.2 Unused signal checks..... 78
21.3 Synthesis smoke test..... 79
22 Adding a New Standard Variant..... 80
22.1 Naming convention..... 81
23 Adding a New Generator Option..... 82
23.1 Backward compatibility..... 83
24 Adding APB5 Parity or Sideband..... 84
24.1 Widening sideband signals..... 85
24.2 Changing parity behavior..... 86
25 Testing an Extension..... 87
25.1 Minimal test plan..... 88
25.2 Example: adding a custom variant test..... 89
25.3 Commit discipline..... 90

List of Figures

Figure 3.1: Lattice Generator Pipeline..... 40

List of Tables

Table 1: Document Properties....9
Table 2: Document Revision History....10
Table 3: Generator Output Artifacts....15
Table 4: Core Generator CLI Options....23
Table 5: Batch Generator CLI Options....24
Table 6: Generation-Time Parameters....27
Table 7: Elaboration-Time Parameters....28
Table 8: PWAKEUP Direction at the Crossbar....31
Table 9: APB5 Parity Check Signals....34
Table 10: Generator Structural Validation Rules....37
Table 11: Generator Address-Map Validation Rules....38
Table 12: Generated Port Naming Convention....61
Table 13: cmd/rsp Fabric Signals....62
Table 14: Sources Requiring Full Regeneration....67
Table 15: Pre-Generated Fixtures....70
Table 16: DV Test List....73

1 Lattice Gas Index

Generated: 2026-10-14

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

2 Document Information

2.1 Lattice Generator Architecture Specification

Document Properties

Property	Value
Document Title	Lattice Generator Architecture Specification
Version	1.0
Date	2026-10-12
Status	Released
Classification	Open Source - MIT License

2.2 Revision History

Document Revision History

Version	Date	Author	Description
1.0	2026-10-12	RTL Design Sherpa	Initial GAS release — verified against `apbx_xbar_generator.py`, `generate_xbars.py`, and emitted fixtures in `rtl/`.
1.2	2026-10-13	seang	Rebrand: component codename **lattice**, directory `lattice-apbx-xbar`; book titles and document number (LATTICE-GAS-001) updated; RTL module names unchanged (`apbx_xbar_*`)

2.3 Document Purpose

This Generator Architecture Specification (GAS) documents the Lattice code generator. It is written for the engineer who must re-run the generator, add a new variant or option, or understand why a generated module looks the way it does.

For the generated hardware, see the **Lattice HAS** and **MAS**. For product requirements, see **PRD.md**.

2.4 Intended Audience

- RTL engineers extending the generator
- Verification engineers regenerating fixtures
- Integrators choosing pre-generated vs custom variants

2.5 Related Documents

- **Lattice HAS** - Hardware Architecture Specification
- **Lattice MAS** - Micro-Architecture Specification
- **PRD.md** - Product requirements
- **CLAUDE.md** - AI development guide

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

3 What the Generator Produces

The Lattice generator (`bin/apbx_xbar_generator.py`) emits a single-file SystemVerilog module for an M×N APB crossbar. The module is parameterized at elaboration time for address width, data width, and base address; the master/slave counts and address-map stride are fixed at generation time.

3.1 Output artifacts

Generator Output Artifacts

Artifact	Description
`apbx_xbar_<M>to<N>.sv`	Core crossbar module.
`apbx_xbar_<M>to<N>_wrap.sv`	Testbench scaffold (only for the four legacy fixtures).
`apbx_xbar_<M>to<N>.f`	Filelist (when generated by an area flow, not by the generator itself).

The generator emits the final form: banner, SPDX header, `reset_defs.svh` include, module header, and complete internal logic. No post-processing is required.

3.2 Architecture in the generated module

The generated module contains:

1. **Master-side APB slave modules** — `apb4_slave` or `apb5_slave` per master, converting APB → cmd/rsp.
2. **cmd/rsp fabric** — address decode, per-slave round-robin arbitration, response routing.
3. **Slave-side APB master modules** — `apb4_master` or `apb5_master` per slave, converting cmd/rsp → APB.

The generator also supports mixed APB4/APB5 ports and optional APB5 parity.

4 When to Regenerate

Regenerate when any of the following change:

- `bin/apbx_xbar_generator.py`
- `bin/generate_xbars.py`
- The `apb4_slave` / `apb4_master` / `apb5_slave` / `apb5_master` boundary modules
- The `arbiter_round_robin` module

The standard regeneration workflow is:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin
python generate_xbars.py        # regenerates all standard variants
into rtl/
```

After regeneration, run the DV suite:

```
cd ../dv/tests
make clean-all && make run-all-gate
```

4.1 Pre-generated vs custom

Always check whether a pre-generated variant exists before generating a custom one. The pre-generated fixtures are the regression baseline; custom variants are for one-off sizes.

5 Generator Entry Points

5.1 Low-level generator

```
python3 apbx_xbar_generator.py --masters 2 --slaves 4 --output
apbx_xbar_2to4.sv
```

CLI flags are documented in the configuration chapter.

5.2 Convenience batch script

```
python3 generate_xbars.py              # all standard variants
python3 generate_xbars.py --masters 3 --slaves 6
python3 generate_xbars.py --masters 4 --slaves 8 --base-addr
0x80000000
```

5.3 Programmatic API

```
from apbx_xbar_generator import generate_apbx_xbar

code = generate_apbx_xbar(
    num_masters=2,
    num_slaves=4,
    base_addr=0x10000000,
    addr_width=32,
    data_width=32,
    output_file="apbx_xbar_2to4.sv",
    slave_size=0x10000,
    master_versions=['apb4', 'apb4'],
    slave_versions=['apb4', 'apb4'],
    name_suffix="",
    enable_parity=False,
)
```

The function returns the generated SystemVerilog as a string and also writes the file when `output_file` is provided.

---

6 CLI and Programmatic API

The generator is invoked either directly for a custom crossbar or through the batch script for the standard family.

6.1 apbx_xbar_generator.py

```
python3 apbx_xbar_generator.py --masters 2 --slaves 4 --output
apbx_xbar_2to4.sv
```

Core Generator CLI Options

Flag	Short	Default	Description
`--masters`	`-m`	required	Number of master interfaces (1-16).
`--slaves`	`-s`	required	Number of slave interfaces (1-16).
`--base-addr`	`-b`	`0x10000000`	Base address of the slave address map.
`--slave-size`		`0x1000`	Address space per slave; must be a power of two and at least 256 bytes.
`--addr-width`	`-a`	32	APB PADDR width.
`--data-width`	`-d`	32	APB PWDATA/PRDATA width.
`--output`	`-o`	`apbx_xbar_<M>to<N>.sv`	Output filename.

The CLI does **not** expose per-port APB5 or parity knobs; those are controlled by the programmatic API and the batch script.

6.2 generate_xbars.py

```
python3 generate_xbars.py                # regenerate all standard
variants
python3 generate_xbars.py --masters 3 --slaves 6
python3 generate_xbars.py --masters 4 --slaves 8 --base-addr
0x80000000 --slave-size 0x10000
```

Batch Generator CLI Options

Flag	Short	Default	Description
`--masters`	`-m`	(all)	Masters for a custom variant.
`--slaves`	`-s`	(all)	Slaves for a custom variant.
`--base-addr`	`-b`	`0x10000000`	Base address for custom variant.
`--slave-size`		`0x10000`	Stride for custom variant (default matches the shipped family).

The batch script explicitly passes `slave_size=0x10000` to the generator so the shipped family uses 64KB windows. Omitting the flag would silently fall back to the generator's 4KB default, which is why the batch script always passes it.

6.3 Programmatic API

```
from apbx_xbar_generator import generate_apbx_xbar

code = generate_apbx_xbar(
    num_masters=2,
    num_slaves=4,
    base_addr=0x10000000,
    addr_width=32,
    data_width=32,
    output_file="apbx_xbar_2to4.sv",
    slave_size=0x10000,
    master_versions=['apb4', 'apb4'],
    slave_versions=['apb4', 'apb4'],
    name_suffix="",
    enable_parity=False,
)
```

The function returns the generated module as a string and writes the file when `output_file` is provided. The `master_versions` and `slave_versions` lists are the only way to request APB5 ports or the mixed 2×2 variant; the CLI is kept narrow because mixed-version generation is normally driven by `generate_xbars.py`.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

7 Parameter Semantics

The generator resolves two classes of parameters: **generation-time** (fixed in the emitted module) and **elaboration-time** (left as `parameter` in the generated SystemVerilog).

7.1 Generation-time parameters

These are baked into the module name, the instance names, and the address map comments.

Generation-Time Parameters

Parameter	Effect in emitted RTL
`num_masters`	Number of master APB interfaces, master-side `apb4_slave`/`apb5_slave` instances, and width of per-slave request/grant vectors.
`num_slaves`	Number of slave APB interfaces, slave-side `apb4_master`/`apb5_master` instances, and number of arbitration blocks.
`base_addr`	Stamped into `BASE_ADDR` default and the address-map comment.
`slave_size`	Stride between slaves; also sets the bit slice used for slave selection. Must be a power of two.
`master_versions` / `slave_versions`	Selects `apb4_*` vs `apb5_*` boundary modules and the presence of sideband/parity pins.
`enable_parity`	Adds APB5 parity ports when the corresponding port is APB5.
`name_suffix`	Appended to the module and file names (e.g., `_mixed`).

7.2 Elaboration-time parameters

The generated module exposes three parameters:

```
module apbx_xbar_2to4 #(
    parameter int ADDR_WIDTH = 32,
    parameter int DATA_WIDTH = 32,
    parameter int STRB_WIDTH = DATA_WIDTH / 8,
    parameter logic [ADDR_WIDTH-1:0] BASE_ADDR = 32'h10000000
) (
```

Elaboration-Time Parameters

Parameter	Typical use
`ADDR_WIDTH`	Widen or narrow PADDR to match a subsystem.
`DATA_WIDTH`	32 for standard APB; 64 for wide APB variants.
`STRB_WIDTH`	Always `DATA_WIDTH / 8`; kept as a derived parameter for readability.
`BASE_ADDR`	Override the base address at instantiation time without regenerating the module. The address-map comment still reflects the generator's default.

The number of masters and slaves is **not** parameterized: a 2×4 module cannot be instantiated as 2×8. Generate a new module instead.

8 Mixed APB4/APB5 and Parity

The generator supports mixed-version ports: any master or slave can be declared APB4 or APB5 independently. The feature is controlled entirely by the `master_versions` and `slave_versions` lists.

8.1 Version list rules

- Each list has length `num_masters` or `num_slaves`.
- Valid values are `'apb4'` or `'apb5'`.
- APB4 ports expose the classic PSEL/PENABLE/PADDR/PWRITE/PWDATA/PSTRB/PPROT/PRDATA/PSLVERR/PREADY set.
- APB5 ports add `PAUSER`, `PWUSER`, `PWAKEUP`, `PRUSER`, and `PBUSER`.

8.2 PWAKEUP direction and forwarding

PWAKEUP is a **Requester** output (AMBA APB IHI 0024E section 3.7.2, Table 3-7). The crossbar sits between two kinds of APB5 interface, so the pin points a different way on each:

PWAKEUP Direction at the Crossbar

Port	Direction at the crossbar	Behaviour
m<i>_apb_PWAKEUP (APB5 requester port)	Input	Driven by the external requester. Parity-checked by the apb5_slave boundary IP when parity is on.
s<j>_apb_PWAKEUP (APB5 completer port)	Output	Driven by the crossbar, which is the Requester on this interface.

Each APB5 completer port drives

```
s<j>_apb_PWAKEUP = <boundary apb5_master's own PWAKEUP>
                 | m<i>_apb_PWAKEUP   for every APB5 requester i
```

- **The boundary term** is the apb5_master IP's registered wake. It is high while a command for that completer is pending or a transfer is in flight, so it leads PSEL by a cycle and holds through PREADY. It covers every transfer, including one from an APB4 requester that has no PWAKEUP at all. 3.7.2 permits a Completer to wait for PWAKEUP before asserting PREADY, so a PSEL without it could deadlock.
- **The requester terms** forward each APB5 requester's wake. Every requester can address every completer here, because there is one address map for all masters. "Any requester that can address this completer" is therefore all of them. The wake is not routed by decode: the address is unknown until PSEL, and 3.7.2 recommends PWAKEUP lead PSEL. The cost is that a requester's wake also raises PWAKEUP on completers it will not access. 3.7.2 permits a wake with no transfer, though it does not recommend one.
- **Glitch-free.** 3.7.2 requires PWAKEUP to be glitch-free and names a glitch-free OR tree as a valid construction. The tree ORs a flop output with requester

PWAKEUP inputs, and those inputs are themselves required to be glitch-free. Nothing is registered, so a requester's wake reaches the completer in the same cycle and keeps its lead over PSEL.

The completer's PWAKEUP is not forwarded back to any requester, because it has no completer-to-requester meaning. The old crossbar treated PWAKEUP as completer-driven and dropped it (the `rsp_pwakeup` and `wakeup_request` boundary pins). Both pins are gone from `apb5_master` and `apb5_slave` (issue #102 item 7).

8.3 Mixed-port sideband behavior

The cmd/rsp fabric between boundary modules is where the protocol conversion happens. Sideband signals are carried only when **both** the source master and the target slave are APB5:

- If the source master is APB5 but the selected slave is APB4, `cmd_pauser` and `cmd_pwuser` are dropped at the slave-side APB4 master.
- If the source master is APB4 but the selected slave is APB5, the APB5 master ties `cmd_pauser` and `cmd_pwuser` to zero.
- Response sideband (`pruser`, `pbuser`) follows the same rule in reverse.

This keeps the fabric simple: there is one cmd/rsp width per direction, and the boundary modules handle expansion or zero-padding.

8.4 Parity (enable_parity=True)

APB5 parity is optional and independent of the version lists. The generator emits parity pins only on APB5 interfaces. The pins are the IHI 0024E chapter 5 check signals. Parity is **odd**: each check bit plus the bits it covers hold an odd number of ones. Wide signals get one check bit per 8 bits (Table 5-1). For a one-bit signal, the check bit is the inversion of the signal.

APB5 Parity Check Signals

Check signal	Width at the crossbar	Covers	Requester port (m<i>_)	Completer port (s<j>_)
PADDRCHK	(ADDR_WIDTH+7)/8	PADDR	input, checked	output, regenerated
PCTRLCHK	1	PPROT, PWRITE (no PNSE is carried)	input, checked	output, regenerated
PSELCHK	1	PSEL (PSELxCHK)	input, checked	output, regenerated
PENABLECHK	1	PENABLE	input, checked	output, regenerated
PWDATACHK	STRB_WIDTH	PWDATA	input, checked	output, regenerated
PSTRBCHK	1	PSTRB	input, checked	output, regenerated
PAUSERCHK	1	PAUSER	input, checked	output, regenerated
PWUSERCHK	1	PWUSER	input, checked	output, regenerated
PWAKEUPCHK	1	PWAKEUP	input, checked	output, ~s<j>_apb_PWAKEUP
PRDATACHK	STRB_WIDTH	PRDATA	output, regenerated	input, checked
PREADYCHK	1	PREADY	output, regenerated	input, checked

Check signal	Width at the crossbar	Covers	Requester port (m<i>_)	Completer port (s<j>_)
PSLVERRCHK	1	PSLVERR	output, regenerated	input, checked
PRUSERCHK	1	PRUSER	output, regenerated	input, checked
PBUSERCHK	1	PBUSER	output, regenerated	input, checked

The user signals are 1 bit on the crossbar surface, so each user check signal is 1 bit. A completer port's PWAKEUPCHK is computed by the crossbar over the OR above. It is not taken from the apb5_master IP, whose own check bit covers only the IP's term, so that output is left unconnected.

Parity does **not** cross the cmd/rsp fabric. The boundary IP deconstructs the APB transfer, checks parity at entry under each signal's Table 5-1 Check Enable term, and regenerates it at exit. Each APB5 requester port exposes m<i>_parity_error_wdata and m<i>_parity_error_ctrl. Each APB5 completer port exposes s<j>_parity_error_rdata and s<j>_parity_error_ctrl. A fabric fault is therefore distinguishable from a slave's own PSLVERR response. When parity is off, the boundary IP's check inputs are tied LOW and its check outputs are left unconnected. The crossbar module then has no check pins at all.

test_apbx_xbar_parity.py generates an all-APB5 2x2 variant with enable_parity=True into its build directory. It checks every check output for odd parity, injects bad PWDATACHK and PRDATACHK and expects the matching error flag, and checks PWAKEUP forwarding from either requester to both completers.

The shipped apbx_xbar_2to2_mixed variant uses APB5 without parity. Parity is reserved for custom subsystems that explicitly request it.

---

9 Validation Rules

The generator validates inputs early and raises `ValueError` before emitting any RTL.

9.1 Structural checks

Generator Structural Validation Rules

Rule	Error message shape
`1 <= num_masters <= 16`	“Number of masters must be 1-16, got N”
`1 <= num_slaves <= 16`	“Number of slaves must be 1-16, got N”
`len(master_versions) == num_masters`	“version list length mismatch”
`len(slave_versions) == num_slaves`	“version list length mismatch”
Each version is `'apb4'` or `'apb5'`	“unknown port version ‘…’”

9.2 Address-map checks

Generator Address-Map Validation Rules

Rule	Rationale
`slave_size` must be a power of two.	Slave selection uses bit slicing after subtracting `BASE_ADDR`.
`slave_size >= 256` (0x100).	Keeps the stride above the smallest practical APB register region.

The generator does **not** check whether the full span (`num_slaves * slave_size`) overflows the address width; that is the caller's responsibility.

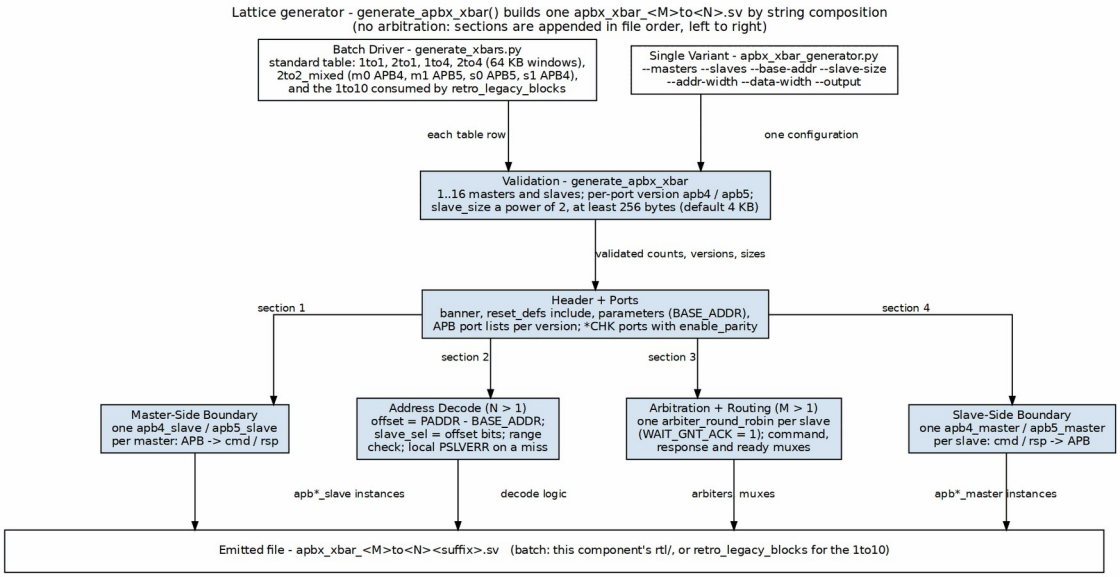
9.3 Parity checks

There is no explicit parity validation beyond the version lists. If `enable_parity=True` and a port is APB4, the generator simply omits parity pins for that port. This is intentional: mixed parity-on/off systems are allowed, and the absence of pins is the safe default.

10 Pipeline Overview

The generator runs in a single Python function (`generate_apbx_xbar`) and emits one SystemVerilog file. There is no separate AST framework; the output is built by string composition.

10.0.1 Figure 3.1: Lattice Generator Pipeline



Lattice Generator Pipeline

Either the batch driver (one call per row of its variant table) or the single-variant command line feeds `generate_apbx_xbar()`, which validates the counts, versions and window size, writes the header and ports, then appends four sections in file order: the master-side boundary IP, the address decode (only when there is more than one slave), the per-slave arbitration and routing (only when there is more than one master), and the slave-side boundary IP. Nothing in the generator arbitrates, so the figure has no trapezium.

10.1 Stages

1. **Validation** — counts, version lists, and `slave_size` checks.
2. **Header emission** — SPDX banner, module name, parameter list, port list, and address-map comment.
3. **Master-side boundary instantiation** — one `apb4_slave` or `apb5_slave` per master with its cmd/rsp nets.
4. **Request decode** — per-master address-to-slave index calculation (offset from `BASE_ADDR`).
5. **Arbitration and routing** — per-slave round-robin arbiter, command mux, response mux, and ready routing.
6. **Slave-side boundary instantiation** — one `apb4_master` or `apb5_master` per slave.
7. **Footer** — `endmodule` and module-name label.

The batch script `generate_xbars.py` loops over the standard variant table and calls the generator once per variant.

---

11 Request Decode

Each master presents a cmd/rsp command after the master-side `apb4_slave`/`apb5_slave` has accepted the APB transaction. The generator decodes the command's `cmd_paddr` to determine which slave (if any) should service it.

11.1 Offset-based slave selection

The slave index is computed from the offset (PADDR - BASE_ADDR), not from raw PADDR bits:

```
m0_cmd_offset    = m0_cmd_paddr - BASE_ADDR;
m0_addr_in_range = (m0_cmd_paddr >= BASE_ADDR) &&
                   (m0_cmd_paddr < (BASE_ADDR + 32'h00040000));
m0_slave_sel     = m0_cmd_offset[17:16];
```

Using the offset prevents the address map from rotating when BASE_ADDR is not aligned to the total span. The subtraction folds to constants at elaboration because BASE_ADDR is a parameter, so it costs no logic.

11.2 Decode-miss handling

If a command falls outside the mapped range, the crossbar must complete the transaction with PSLVERR rather than holding PREADY low forever. The generator emits a per-master r_m<N>_decerr_pending flop:

- When cmd_valid && cmd_ready && !addr_in_range, the pending flag is set.
- The master-side cmd_ready is asserted only while the pending flag is clear.
- A locally generated response (rsp_valid = 1, rsp_pslverr = 1) is returned on the next cycle and cleared when rsp_ready is asserted.

This guarantees that an unmapped access completes as a recoverable error instead of wedging the external master.

11.3 Single-master simplification

When `num_masters == 1`, no arbitration is needed. The request decode still creates the per-slave `cmd_valid` signals directly from the slave-select result, and the decode-miss logic remains.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

12 Arbitration and Routing

For each slave, the generator emits an independent round-robin arbiter that selects among the masters requesting that slave.

12.1 Per-slave request vector

```
s0_arb_request[0] = m0_cmd_valid && m0_addr_in_range && (m0_slave_sel
== 2'd0);
s0_arb_request[1] = m1_cmd_valid && m1_addr_in_range && (m1_slave_sel
== 2'd0);
```

When `num_slaves == 1`, the slave-select check is omitted.

12.2 Round-robin arbitration

The generator instantiates `arbiter_round_robin` with `WAIT_GNT_ACK=1` so the grant is locked until the transaction completes:

```
arbiter_round_robin #(
    .CLIENTS(2),
    .WAIT_GNT_ACK(1)
) u_s0_arbiter (...);
```

The grant-ack vector is driven by `rsp_valid && rsp_ready` on the selected slave port, ensuring the grant persists across the entire APB access.

12.3 Command routing

A combinational `case (1'b1)` mux routes the granted master's cmd fields to the slave-side cmd nets. APB5 sideband is carried only when both the master and the slave are APB5.

12.4 Response routing

The generator registers each master's slave selection (`r_m<N>_slave_sel`) when a command is accepted. On the response path, the registered slave index selects which slave's `rsp_*` signals are returned to the master. The registered selection keeps the response aligned with the command even if a new command arrives before the first response completes.

12.5 Ready routing

- Master `cmd_ready` is asserted only by the slave selected by the current command.
- Slave `rsp_ready` is asserted only by the master currently holding the grant for that slave.
- The decode-miss path bypasses the slave arbiters and completes locally.

13 Boundary Module Instantiation

The generated module instantiates proven boundary modules from the `rtl/common/` and `rtl/amba/` libraries rather than reimplementing APB protocol logic inline.

13.1 Master-side modules

For each master `m`:

- `apb4_slave` when `master_versions[m] == 'apb4'`.
- `apb5_slave` when `master_versions[m] == 'apb5'`.

Parameterization:

```
apb4_slave #(
    .ADDR_WIDTH(ADDR_WIDTH),
    .DATA_WIDTH(DATA_WIDTH),
    .STRB_WIDTH(STRB_WIDTH),
    .PROT_WIDTH(3)
) u_apb4_slave_m0 (...);
```

APB5 variants add user widths of 1 and, when parity is enabled, `ENABLE_PARITY=1`.

13.2 Slave-side modules

For each slave `s`:

- `apb4_master` when `slave_versions[s] == 'apb4'`.
- `apb5_master` when `slave_versions[s] == 'apb5'`.

The same width and sideband parameter rules apply.

13.3 Parity tie-offs

When parity is disabled on an APB5 port, the `*CHK` input pins are tied to zero and the `*CHK` output and `parity_error_*` pins are left unconnected. This lets the same `apb5_slave`/`apb5_master` modules be reused with or without parity without conditional module selection.

13.4 PWAKEUP wiring

An APB5 master port's PWAKEUP input goes to the `apb5_slave` boundary IP, which uses it only for the PWAKEUPCHK check. Each `apb5_master` boundary IP drives its own registered wake onto an internal net, `w_s<s>_ip_pwakeup`. The generator then emits `assign s<s>_apb_PWAKEUP = w_s<s>_ip_pwakeup | m<i>_apb_PWAKEUP ...`, with one term for every APB5 master. With parity on, it also emits `assign s<s>_apb_PWAKEUPCHK = ~s<s>_apb_PWAKEUP`. The IP's own `m_apb_PWAKEUPCHK` is left unconnected, because it covers only the IP's term.

13.5 1×1 passthrough

When `num_masters == 1` and `num_slaves == 1`, the fabric reduces to simple `assign` statements between the single master-side and single slave-side cmd/rsp nets. No arbiter is instantiated.

14 Directory Layout

The generator writes a single `.sv` file per invocation. The standard family is organized as follows:

```
lattice-apbx-xbar/
├── bin/
│   ├── apbx_xbar_generator.py      # low-level generator
│   └── generate_xbars.py           # batch generator
├── rtl/
│   ├── apbx_xbar_1to1.sv
│   ├── apbx_xbar_2to1.sv
│   ├── apbx_xbar_1to4.sv
│   ├── apbx_xbar_2to4.sv
│   └── apbx_xbar_2to2_mixed.sv
└── dv/
    └── tests/                      # cocotb test suite
```

The Retro Legacy Blocks 1-to-10 variant is emitted outside this tree because it is consumed by another component:

```
retro_legacy_blocks/rtl/apbx_xbar/apbx_xbar_1to10.sv
```

This external placement is intentional; it prevents the RLB crossbar from drifting away from the family fixes.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

15 Module Header and Parameters

Every generated file begins with the standard RTL Design Sherpa banner and includes `reset_defs.svh`.

```
// SPDX-License-Identifier: MIT
// SPDX-FileCopyrightText: 2024-2025 sean galloway
//
// RTL Design Sherpa ...
//
// Module: apbx_xbar_2to4
// Purpose: Apbx Xbar 2to4 module
//
// Documentation: docs/markdown/rtl-amba/index.md
// Subsystem: amba
//
// Author: sean galloway
// Created: 2025-10-18

`timescale 1ns / 1ps
`include "reset_defs.svh"
```

15.1 Address-map comment

Immediately before the module declaration, the generator emits a comment block documenting the slave map:

```
//   Slave 0: [0x10000000, 0x1000FFFF] (64KB)
//   Slave 1: [0x10010000, 0x1001FFFF] (64KB)
//   ...
```

This comment is derived from `base_addr` and `slave_size` and is the authoritative map for integrators.

15.2 Port naming convention

Generated Port Naming Convention

Direction	Naming
Master-side inputs	`m<N>_apb_<signal>`
Master-side outputs	`m<N>_apb_<signal>`
Slave-side outputs	`s<N>_apb_<signal>`
Slave-side inputs	`s<N>_apb_<signal>`

Parity error outputs (APB5 with parity) use `m<N>_parity_error_wdata` / `m<N>_parity_error_ctrl` on the master side and `s<N>_parity_error_rdata` / `s<N>_parity_error_ctrl` on the slave side.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

16 cmd/rsp Fabric

Between the APB boundary modules, the generated module implements a command/response fabric with the following signals per master and per slave:

cmd/rsp Fabric Signals

Signal	Direction from master-side	Width	Meaning
`cmd_valid`	master → fabric	1	Command available.
`cmd_ready`	fabric → master	1	Command accepted.
`cmd_pwrite`	master → fabric	1	Write (1) or read (0).
`cmd_paddr`	master → fabric	`ADDR_WIDTH`	Address.
`cmd_pwdata`	master → fabric	`DATA_WIDTH`	Write data.
`cmd_pstrb`	master → fabric	`STRB_WIDTH`	Byte strobe.
`cmd_pprot`	master → fabric	3	Protection.
`cmd_pauser`	master → fabric	1	APB5 sideband (only on APB5 master).
`cmd_pwuser`	master → fabric	1	APB5 sideband (only on APB5 master).
`rsp_valid`	fabric → master	1	Response available.
`rsp_ready`	master → fabric	1	Response accepted.
`rsp_prdata`	fabric → master	`DATA_WIDTH`	Read data.
`rsp_pslverr`	fabric → master	1	Slave error.

Signal	Direction from master-side	Width	Meaning
`rsp_pruser`	fabric → master	1	APB5 sideband (only on APB5 master).
`rsp_pbuser`	fabric → master	1	APB5 sideband (only on APB5 master).

The slave-side cmd/rsp signals have the same names and directions reversed relative to the slave-side APB master modules.

16.1 Fabric width invariance

The cmd/rsp width is fixed regardless of mixed APB4/APB5 ports. APB4 ports simply do not drive or receive the sideband signals; APB4 slaves tie them to zero and APB4 masters ignore them. This avoids parameterized struct widths inside the fabric.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

17 Wrapper Scaffolds

The generator emits only the core crossbar module. Wrapper scaffolds for testbench convenience exist only for the four legacy fixtures and are **not** regenerated by `generate_xbars.py`. If a new variant needs a wrapper, create it by hand in the DV area or add a wrapper emission stage to the generator.

17.1 Filelist convention

Area flows that consume the generated modules typically reference the RTL directly rather than a generated `.f` file. An example filelist for the standard variants would contain:

```
rtl/amba/apb4/xbar/apbx_xbar_2to4.sv
rtl/common/arbiter_round_robin.sv
rtl/common/apb4_slave.sv
rtl/common/apb4_master.sv
```

The exact dependencies are determined by the variant (APB5 ports add `apb5_slave.sv` and `apb5_master.sv`). The component-level `Makefile` or area regression flow maintains the canonical filelist.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

18 Full Regeneration Rule

Any change to the files listed below must be followed by a full regeneration of every standard variant and a full regression run.

Sources Requiring Full Regeneration

Source	Why it forces regeneration
`bin/apbx_xbar_generator.py`	Every emitted line comes from this file.
`bin/generate_xbars.py`	Drives the variant table and default parameters.
`rtl/common/apb4_slave.sv` / `apb5_slave.sv`	Boundary module behavior changes propagate into every master-side instance.
`rtl/common/apb4_master.sv` / `apb5_master.sv`	Same for slave-side instances.
`rtl/common/arbiter_round_robin.sv`	Arbitration timing or fairness changes affect every multi-master variant.

18.1 Regeneration command

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin
python generate_xbars.py
```

After regeneration, the only expected `git diff` should be in `rtl/*.sv` and, if the RLB variant was affected, `retro_legacy_blocks/rtl/apbx_xbar/apbx_xbar_1to10.sv`.

18.2 Do not hand-edit generated files

The `.sv` files in `rtl/` are generator output. Hand-edits will be overwritten on the next regeneration and are not reviewable as source changes. Fix the generator instead.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

19 Pre-Generated Fixtures

The component ships with five pre-generated fixtures:

Pre-Generated Fixtures

File	Masters	Slaves	Notes
`apbx_xbar_1to1.sv`	1	1	Simple passthrough; no arbiter.
`apbx_xbar_2to1.sv`	2	1	Arbitration only, no address decode.
`apbx_xbar_1to4.sv`	1	4	Address decode only, no arbitration.
`apbx_xbar_2to4.sv`	2	4	Full crossbar: decode + arbitration.
`apbx_xbar_2to2_mixed.sv`	2	2	APB4/APB5 mixed-port boundary test.

These fixtures are the regression baseline. They are generated with:

- `base_addr = 0x10000000`
- `slave_size = 0x10000` (64KB windows)
- `addr_width = 32`, `data_width = 32`

The mixed variant uses `master_versions=['apb4','apb5']` and `slave_versions=['apb5','apb4']`.

19.1 When to add a new fixture

Add a new standard fixture only when:

1. A new size is required by multiple subsystems, or
2. A new port-version combination needs continuous regression coverage.

Add the new entry to the `variants` or `mixed` table in `generate_xbars.py`, regenerate, and add a matching cocotb test.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

20 DV Suite

The component-level DV suite lives in `dv/tests/` and uses cocotb.

20.1 Test list

DV Test List

Test	Focus
`test_apbx_xbar_1to1.py`	Passthrough behavior.
`test_apbx_xbar_2to1.py`	Two masters contend for one slave.
`test_apbx_xbar_1to4.py`	Single master accesses multiple slaves by address.
`test_apbx_xbar_2to4.py`	Full decode + arbitration stress.
`test_apbx_xbar_2to2_mixed.py`	APB4/APB5 sideband conversion.
`test_apbx_xbar_timing.py`	PREADY backpressure and timing.

20.2 Running the suite

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/dv/tests
make clean-all && make run-all-gate
```

The `run-all-gate` target exercises every standard fixture through the gate-level-style regression flow. After a generator change, this must pass with zero failures before the regenerated RTL is committed.

20.3 Adding a new test

1. Add a new `test_apbx_xbar_<variant>.py` file in `dv/tests/`.
2. Reference the matching generated module in the test's `Makefile` or parameter list.
3. Add the test name to the area regression list (component `Makefile` or `tests.mk`).

---

21 Lint Checks

The generated RTL must pass the project's lint flow. The most common lint concerns for generated crossbars are width mismatches and unused signals.

21.1 Width checks

- `slave_addr_bits` is `ceil(log2(num_slaves))` and is at least 1.
- `slave_offset_bits` is `log2(slave_size)` and is used to slice the offset.
- The `BASE_ADDR` parameter is `ADDR_WIDTH` bits.
- APB5 sideband signals are 1-bit logic. Check vectors are `(ADDR_WIDTH+7)/8` bits for `PADDRCHK`, `STRB_WIDTH` bits for `PWDATACHK`/`PRDATACHK`, and 1 bit for every other `*CHK` signal.

21.2 Unused signal checks

When parity is disabled on an APB5 port, the generator leaves some output pins unconnected. The lint waiver for these is part of the boundary module, not the generated crossbar. Ensure the waiver file or lint command set covers `apb5_slave` and `apb5_master` tie-offs.

21.3 Synthesis smoke test

As a minimum sanity check, run a synthesis tool on the 2×4 variant after any generator change. The expected result is clean elaboration with no latch inference and no combinational loops.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

22 Adding a New Standard Variant

To add a new pre-generated fixture size:

1. Open `bin/generate_xbars.py`.
2. Add the `(masters, slaves)` tuple to the `variants` list for plain APB4, or add a `mixed` entry for a new version combination.
3. Add a matching cocotb test in `dv/tests/`.
4. Regenerate:

```
cd projects/components/fabric-gen-ip/lattice-apbx-xbar/bin
python generate_xbars.py
```

5. Run the DV suite:

```
cd ../dv/tests
make clean-all && make run-all-gate
```

6. Commit the generator change, the new RTL file, and the new test together.

22.1 Naming convention

- Plain variants: `apbx_xbar_<M>to<N>.sv`
- Mixed-version variants: `apbx_xbar_<M>to<N>_<suffix>.sv`

The suffix must be alphanumeric with underscores and must be reflected in the `name_suffix` argument.

---

23 Adding a New Generator Option

To add a scalar option that affects all variants (for example, a new default `slave_size`):

1. Add the argument to `apbx_xbar_generator.py`'s `argparse` block and pass it through `generate_apbx_xbar`.
2. Add the same argument to `generate_xbars.py` if users should be able to override it for the standard family.
3. Document the option in this GAS, the HAS, and the PRD if it changes the hardware behavior.
4. Add validation near the top of `generate_apbx_xbar`.
5. Regenerate and run the full DV suite.

23.1 Backward compatibility

Default values must preserve existing generated output. If the new option changes the default address map or port list, introduce it as an opt-in flag first and migrate the standard variants in a separate commit.

RTL Design Sherpa · Learning Hardware Design Through Practice Documentation Index · MIT License

---

24 Adding APB5 Parity or Sideband

The APB5 sideband and parity machinery is already present in the generator. Extending it usually means widening a sideband signal or changing how parity is computed.

24.1 Widening sideband signals

The generator hard-codes 1-bit user signals (`AUSER_WIDTH=1`, `WUSER_WIDTH=1`, etc.). To support wider user fields:

1. Add a new generator option (e.g., `user_width`).
2. Pass the width through to the `apb5_slave`/`apb5_master` parameterization.
3. Update the fabric cmd/rsp declarations to carry the wider fields.
4. Update the port list generation to declare the wider APB5 pins.

24.2 Changing parity behavior

The check signals are the AMBA APB (IHI 0024E) chapter 5 `*CHK` set. They use odd parity, with one bit per 8 bits of the covered signal. The boundary modules check them on entry and generate them on exit, and the crossbar only routes them. The one exception is a completer port's `PWAKEUPCHK`, which the crossbar computes as `~PWAKEUP` over its own wake OR (see *Mixed APB4/APB5 and Parity*). To change parity width or algorithm:

1. Modify `apb5_slave` and `apb5_master` to support the new scheme.
2. Update the crossbar generator's parity vector widths and tie-offs accordingly.
3. Extend `dv/tests/test_apbx_xbar_parity.py`, which generates a parity-enabled variant, checks every check output for odd parity, and injects check-bit errors against the `parity_error_*` outputs.

Remember that parity is checked and regenerated at each boundary. It never crosses the cmd/rsp fabric.

---

25 Testing an Extension

Every generator change must be exercised by at least one generated fixture and one DV test.

25.1 Minimal test plan

1. **Generator unit test** — call `generate_apbx_xbar` with the new option and verify the emitted string contains the expected constructs.
2. **Fixture regeneration** — run `generate_xbars.py` and verify no unexpected diffs appear in existing fixtures.
3. **Simulation test** — add or extend a cocotb test that exercises the new behavior.
4. **Lint/synthesis smoke** — run the lint flow on the affected variants.

25.2 Example: adding a custom variant test

```
# dv/tests/test_apbx_xbar_3to6.py
from apbx_xbar_test_base import ApbxXbarTestBase

class TestApbxXbar3to6(ApbxXbarTestBase):
    def test_basic_access(self):
        self.dut = cocotb.top
        # exercise all 3 masters against all 6 slaves
```

The test must be added to the `run-all-gate` target or an equivalent regression list.

25.3 Commit discipline

Commit generator source changes, regenerated RTL, and DV tests in a single logical change. Do not commit hand-edited generated files.